

Schedule and Generated Work

Last updated: 2026-05-11

The scheduler generates work snapshots for assets based on their estimate matrix values and writes those snapshots back to the studio's source tool (Jira or Airtable). Generated work is ArtHound's layer on top of source work, it provides structured, estimate-grounded task records that can be reconciled against what the source tool actually contains.

Core Distinction: Source Work vs Generated Work

Source work (`replicated_work`), synced copies of work items that already exist in the source tool. ArtHound reads these but does not own them.

Generated work (`generated_work`), snapshots created by ArtHound's scheduler, representing the recommended task breakdown for an asset. ArtHound owns these. They are written to the source tool on generation, but the source tool is the operational system of record after that point.

Generated work is always filtered by `deleted_at IS NULL` in normal queries. Soft deletion is used so historical snapshots are preserved for variance analysis.

Generation Flow

```
POST /api/schedule/generate      ← single asset
POST /api/schedule/generate-bulk ← multiple assets, parallel
├── Resolve estimate matrix for asset's variable values
├── Apply topological step order from workflow_step_dependencies
├── Compute start/end dates (sequential from a base date; respects step deps)
├── Write snapshot rows to generated_work
└── Write records to source tool (Jira or Airtable)
    ├── Airtable: create_record() per step
    └── Jira:      create_issue() per step + create_issue_link() to asset
```

Write-back: Airtable

For each workflow step, `create_record(table_id, fields, qualifier_defaults)` is called with: - Task name, craft, estimate, start/end dates - Link field resolving the record back to the source asset (via `rel_field_id` from `source_entity_definitions`) - `qualifier_defaults` merged in to satisfy PAW qualifier fields (see [Sync Layer, Qualifiers](#))

Write-back: Jira

For each workflow step, `create_issue(fields, qualifier_defaults)` creates a Jira issue, then `create_issue_link("relates to", asset_jira_key, new_issue_key)` links it to the asset issue. Custom field mappings from `source_entity_definitions` are applied.

Known gaps in Jira write-back: - Sub-task creation requires `parent.key` which is not always available - If an asset's `_jira_key` is missing, the issue is created but the link step silently returns 0 (not surfaced to the UI) - Per-item failure detail is not returned to the frontend on bulk generation

Preview Without Saving

`POST /api/schedule/preview` runs the same generation logic but does not write to `generated_work` or the source tool. Returns the proposed task list with dates and estimates for the studio to review before committing.

Viewing Generated Work

`GET /api/schedule/work-local?canonicalAssetId={id}` returns generated work snapshots for a single asset. This is what the Work tab in the Asset Viewer shows when the "ArtHound" source is selected.

Generated work items returned include: `work_name`, `craft`, `estimate_days`, `start_date`, `end_date`, `workflow_step_id`, `variable_values`.

Reconcile Work

`POST /api/schedule/reconcile-work` soft-deletes generated work snapshots that no longer have a corresponding active source record. This handles the case where a source tool task was manually deleted after ArtHound generated it.

The reconciler: 1. Queries all non-deleted generated work for the studio 2. Cross-references against current `source_record_id` values in `replicated_work` 3. Sets `deleted_at = now()` on any generated work row whose source record is gone

This is an idempotent operation and can be run at any time.

Database Schema

generated_work

```
id                uuid PK
canonical_asset_id  uuid → canonical_assets
studio_id         uuid → studios
source_type       text (airtable | jira)
source_record_id   text (the ID of the record created in the source tool)
work_name         text
workflow_step_id   uuid → workflow_steps
craft            text
estimate_days      numeric
variable_values    jsonb (snapshot of the asset values used for this estimate)
start_date        date
end_date          date
generated_at       timestampz
deleted_at        timestampz (soft delete; NULL = active)
```

Always filter `deleted_at IS NULL` unless querying history.

RLS: studio-scoped (vendors cannot see generated work directly; they see dispatched asset data via the payload system).

API Reference

All endpoints are under `/api/schedule` . Studio-only.

Method	Path	Description
GET	<code>/work-local</code>	Generated work snapshots for an asset (<code>?canonicalAssetId=</code>)
POST	<code>/preview</code>	Preview generation without saving
POST	<code>/generate</code>	Generate and write for one asset
POST	<code>/generate-bulk</code>	Generate and write for multiple assets (parallel)
POST	<code>/reconcile-work</code>	Soft-delete orphaned generated work snapshots

Known Gaps

Jira sub-task linking: sub-task creation requires `parent.key` from the parent Jira issue. This is not always available when generating, so sub-tasks are created as regular issues and linked instead.

Missing `_jira_key` warning not surfaced, if an asset's Jira key is unavailable at generation time, the write-back proceeds but the issue link step fails silently. The UI receives a success count without knowing which items failed to link.

No per-item failure detail on bulk: `POST /generate-bulk` returns an aggregate success/failure count, not a per-asset breakdown. If three out of ten assets fail source write-back, the studio cannot see which three without checking the source tool.

Source write-back is fire-and-forget: if the source tool write-back fails (network error, rate limit), the `generated_work` snapshot has already been written. The snapshot exists in ArtHound with no corresponding source record. No retry or cleanup is performed automatically.